



THE LAB CODER

Advanced Notes Part 2 (update 18 Aug 2026)

Notes: <http://tinyurl.com/TheLabAdvanced2>

Content

Page	
3	[3040] <u>Try-Exception</u>
8	[3041] <u>Function with parameters</u>
20	[3041] <u>Function with return values</u>
28	[3042] <u>2D List</u>
35	[3043] <u>Dictionary</u>
45	[3044] <u>File I/O (read and write)</u>
55	[3045] <u>File I/O (file pointer)</u>
59	[3046] <u>Turtle</u>

Click on titles

TRY - EXCEPTION

Try-Except

When a code performs an invalid operation (eg. trying to cast a variable containing letters to integer), it will terminate the program with an error code. Errors commonly comes from user entering invalid input data. Instead of ending the program abruptly, you can handle the error in your code. *Try-except* allows you to do that.

try:

Indent with tab

run codes that might cause error here...

except:

run codes to respond to error here...

Try-Except

Example:

```
num = input("Enter an integer: ")
```

```
try:
```

```
    num = int(num) ← if this causes an error, it jumps to except  
    print("You did it!")
```

```
except:
```

```
    print("Oh dear, I cannot cast that!")
```

```
print("The program continues...") ← whether there is an error or not,  
                                  the program will continue
```

Screen output

```
Enter an integer: 10  
You did it!  
The program continues...
```

```
Enter an integer: ten  
Oh dear, I cannot cast that!  
The program continues...
```

Try-Except

It is also valid to place your codes in this way. It will have the same results. Whichever code causes an error will make it jump to *except* immediately.

```
num = input("Enter an integer: ")  
  
try:  
    num = input("Enter an integer: ")  
    num = int(num)  
    print("You did it!")  
except:  
    print("Oh dear, I cannot cast that!")  
  
print("The program continues...")
```

this code can be shifted in as well



Try-Except

You can make your program robust by enforcing user to key in valid data. Below is an example.

```
while True:
    try:
        num = input("Enter an integer from 1 to 10: ")
        num = int(num)
        if num >= 1 and num <= 10:
            break
        else:
            print("Number out of range. Try again.")
    except:
        print("Invalid entry. Try again.")
print("You did it!")
```

Screen output

```
Enter an integer from 1 to 10: 0
Number out of range. Try again.
Enter an integer from 1 to 10: five
Invalid entry. Try again.
Enter an integer from 1 to 10: 5
You did it!
```



FUNCTION *with parameters*

Must I code using functions?

You are learning coding to understand and write real-world applications. Writing code as functions is **NOT OPTIONAL**. Real applications are complex, and functions are essential for debugging, teamwork, maintenance, readability, and reuse. Skipping them may work for small exercises, but it is unsustainable in professional practice.

It is **INCORRECT PRACTICE** to write your entire program in one main block and only later try to rearrange it into functions. The correct approach is to design your code structure with functions from the start and progressively build them up. This approach helps you break down programs, no matter how complex, and allows you to develop and test them progressively.

The curriculum will begin with small functions. Over time, your main code will shrink and naturally evolve into a structured program built around functions.

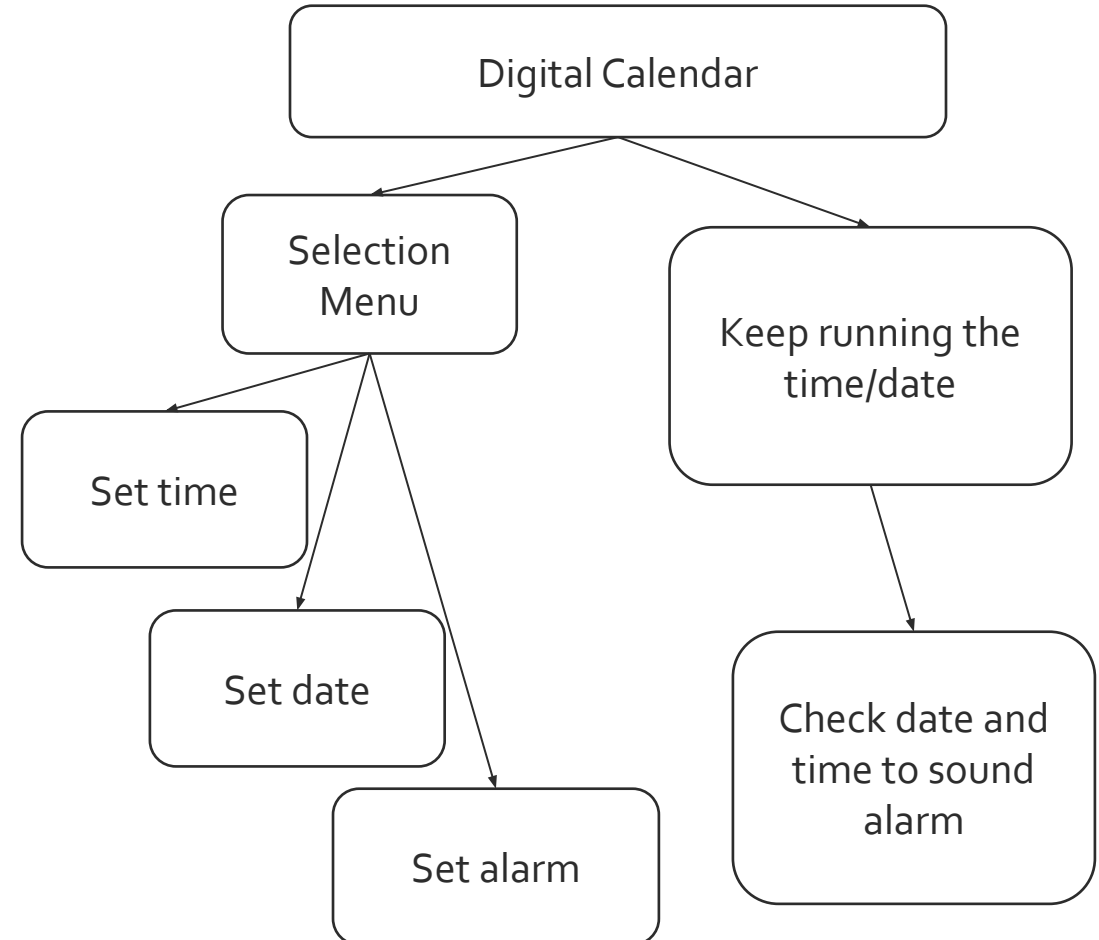
Function

A function is also called a **method**. It is group of codes that can be executed whenever needed simply by calling the function name. A function can also call other functions.

When do you create functions? As a guide, these are some common reasons:

1. To recycle group of codes that are executed repeatedly.
2. To help in designing your program by breaking into smaller tasks, each task can be a function or a group of functions.
3. To make bigger program easier to read, debug and maintain/update.
4. To enable teamwork by distributing tasks in the form of creating various functions.

Example of how you can break your program into tasks which can be written as functions



Function

To build a function, firstly, you need to define the function. You give it any name following the same rules of naming a variable. Your program can then call it anytime (and any number of times) to execute the codes in it.

defining the function. function will not execute until it is called

def myfunction (optional parameters):

codes here

Indent with tab

calling the function

myfunction()

when the function is called, it will program will go into the function to run the codes. When done, it will return back to the same place to resume other codes.

Function

In a program file, the top section is where you place all the imports, followed by all the function definitions and then the "main" codes where the program begin execution.

We call the section where your code starts running the **main**

program file

library imports
import ...

all function definitions
define function1
define function2
define function3
...

main
Execution of your
codes starts here

Function

Simple function

A simple function has no input parameters nor any return value. That means it can perform its task without needing any information provided nor send back any results when it is called.

Example

```
# print hello world
def myfunction():
    print("Hello World!")
```

```
# main
myfunction()
```

Name a function in any style as long as you follow the variable naming rules.

For examples, you can call it ***myfunction***, ***MyFunction***, ***myFunction*** or ***my_function***. Python programmers like to use underscores to separate multiple words for easy reading. They are usually named according to the task they perform.

Screen output

Hello World!

Function

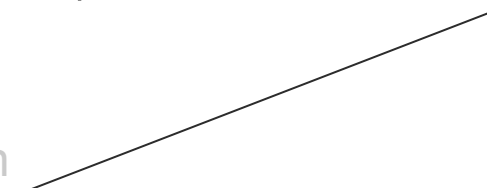
Function accessing variables in main

A function can search for variables in the main. The variables are termed **Global** variables.

Example

```
# print a variable  
def print_variable():  
    print("The name is", name)
```

```
# main  
name = "Barney"  
print_variable()
```



Screen output

The name is Barney

Function

Function with parameters

It is good practice to use **Local** variables for modular design of functions instead of **Global** variables. Function can have input parameters. They are local variables which exists only when the function is called.

Example

display area of a square (provide the length)

```
def print_area(length):
```

```
    print("Area of square is", length*length, "sqm")
```

main

```
print_area(4)
```

The value 4 is passed to *length*. *length* is a **local variable** which exists only within the function. Once the function has completed running its codes, it will cease to exist and recreated upon the next call.

We call the value 4 the **argument** and the function variable *length* the **parameter**.

Screen output

Area of square is 16 sqm

PYTHON CODING

FUNCTION WITH PARAMETERS

Function

Function with parameters

You can also pass **variable** values (also called arguments) to function parameters.

Example

display area of a square (provide the length)

```
def print_area(length):  
    print("Area of square is", length*length, "sqm")
```

main

sqrlength = 4

print_area(**sqrlength**)

Value of *sqrlength*, which is 4, is passed
to variable *length* of the function

Screen output

Area of square is 16 sqm

PYTHON CODING

FUNCTION WITH PARAMETERS

Function

Function with parameters

You can have multiple parameters in your function. Separate them with commas.

Example

display the area of a rectangle (provide the length and breadth)

```
def print_area(length , breadth):
```

```
    print("Area of rectangle is", length*breadth, "sqm")
```

main

```
rect_length = 4
```

```
rect_breadth = 5
```

```
print_area(rect_length, rect_breadth)
```

the values are passed to the
parameters in order

Screen output

Area of square is 16 sqm



PYTHON CODING

FUNCTION WITH PARAMETERS

Function

Function with parameters

You can use the same variable name in the main and in your functions. They are **different** variables even though they have the same name.

Example

display the area of a square

```
def print_area(length):  
    print("Area of square is", length*length, "sqm")
```

main

length = 4

print_area(**length**)

Value of *length* in *main*, which is 4, is passed to variable *length* of the function.

Screen output

Area of square is 16 sqm

Function

Function with parameters

You can create variables within a function. They are created every time the function is called and you will be gone once the function call ends. Hence, you can only use them within the function.

Example

display the area of a square

```
def print_area(length):
```

```
    area = length*length
```

```
    print("Area of square is", area, "sqm")
```

main

```
sqlength = 4
```

```
print_area(sqlength)
```

variable **area** is created every time the function is called and will disappear when the function ended.

Screen output

Area of square is 16 sqm

TypeError:

You when receive the a type error message pertaining to function as shown below, it means the number of arguments provided while calling the function doesn't match its number of parameters.

TypeError: func() missing n required positional argument
TypeError: func() takes n positional arguments but m was given

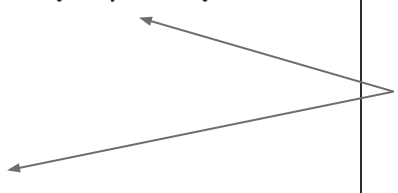
Below shows an example:

```
def func(a, b):
```

```
    ...
```

```
func(1)
```

mismatched number of arguments and parameters



FUNCTION *with return values*

Function

Function with return value

Function can also perform some operations and return the result.
The calling program will then have to receive the result.

Example

calculate area of a rectangle and return the result

```
def find_area(length, breadth):  
    area = length * breadth  
    return area
```

The calculated area is returned and saved to variable *result*.
If you forget a return command, *result* will be a value **None** (*meaning no value*)

main

```
result = find_area(2, 3)  
print("Area of rectangle is", result, "sqm")
```

Screen output

Area of rectangle is 6 sqm

PYTHON CODING

FUNCTION WITH RETURN VALUES

Function:

Function with multiple return values

Function can also return *multiple* values separated by commas.
You can receive them with same number of variables when calling the function.

Example

calculate the area and perimeter of a rectangle and return the results

```
def find_area_perimeter(length, breadth):
```

```
    area = length * breadth
```

```
    perimeter = 2 * (length + breadth)
```

```
    return area, perimeter
```

main

```
rect_area, rect_perimeter = find_area_perimeter(2, 3)
```

```
print("Area of rectangle is", rect_area, "sqm")
```

```
print("Perimeter of rectangle is", rect_perimeter, "m")
```

the return values will be passed to the variables in order

Screen output

```
Area of rectangle is 6 sqm
Perimeter of rectangle is 10 m
```



PYTHON CODING

FUNCTION WITH RETURN VALUES

Function:

Function with multiple return values

You can receive multiple returned results with a single variable. The variable will become a tuple. A tuple is like a list, except that when created, it cannot be amended. You can cast it to list as shown below.

Example

calculate the area and perimeter of a rectangle and return the results

```
def find_area_perimeter(length, breadth):
```

```
    area = length * breadth
```

```
    perimeter = 2 * (length + breadth)
```

```
    return area, perimeter
```

main

the return values will be passed to the variable as a tuple

```
results = find_area_perimeter(2, 3)
```

```
print("Results as tuple:", results)
```

```
results = list( results )
```

```
print("Results as list:", results)
```

```
print("Area of rectangle is", results[0], "sqm")
```

```
print("Perimeter of rectangle is", results[1], "sqm")
```

Screen output

Results as tuple: (6, 10)

Results as list: [6, 10]

Area of rectangle is 6 sqm

Perimeter of rectangle is 10 m



Function:

Once a *return* is executed in a function, it will *leave* the function and the function call ended.

Example

check if x is bigger than y

```
def is_bigger(x, y):
```

```
    if x > y:
```

```
        print(x, "is bigger than", y)
```

```
        return True
```

← once a return is executed, the function call will end and the remaining codes in the function will not be run

```
    print(x, "is not bigger than", y)
```

```
    return False
```

main

```
result = is_bigger(5, 4)
```

```
print(result)
```

Screen output

```
5 is bigger than 4
True
```

Passing Value to Function Parameter:

A function parameter or variable created inside a function can have the same name as a variable outside the but they will be treated as different variables. Changing the one inside a function will not affect the one outside. Variables in the main are call **global variables** and those in a function are called **local variables**.

Example

Function (has a parameter **x**)

```
def change(x):  
    x = 1  
    print("Inside function, x is changed to", x)
```

This **x** is not the same **x** as the main variable although they have the same name. Any change will not affect the **x** outside the function.

main (has a variable **x**)

```
x = 0  
print("Before calling function, x is", x)  
change(x)  
print("After calling function, x is still", x)
```

Screen output

```
Before calling function, x is 0  
Inside function, x is changed to 1  
After calling function, x is still 0
```

Passing Address to Function Parameter:

When passing a variable that contains a group of values (eg. a **list**) to a function parameter, a memory address of where the values are stored is being passed instead. Changing the values inside the function will affect the original variable since they go to the same address to read and change the values.

Example

Function has a parameter **listy**

```
def change(listy):  
    listy[0] = 1  
    print("Inside function, listy is changed to", listy)
```

main has a variable **listx**

```
listx = [7, 8, 9]  
print("Before calling function, listx is", listx)  
change(listx)  
print("After calling function, listx is", listx)
```

listx passed a memory address to **listy**. When **listy** change the value at that address, **listx** will see the change



Memory address: 9223808113
7
8
9

Screen output

```
Before calling function, listx is [7, 8, 9]  
Inside function, listy is changed to [1, 8, 9]  
After calling function, listx is [1, 8, 9]
```

Function:

A function can also call other functions. This allows you to break down large task into smaller tasks.

Example

function to find perimeter of a rectangle

```
def find_perimeter(x, y):  
    return 2 * (x + y)
```

function to find area of a rectangle

```
def find_area(x, y):  
    return x * y
```

function to find area and perimeter of a rectangle

```
def calculate(x, y):  
    area = find_area(x, y)  
    perimeter = find_perimeter(x, y)  
    return area, perimeter
```

call the method and display the returned value

```
result = calculate(4, 5)  
print(result)
```

call other functions

Screen output

20 18

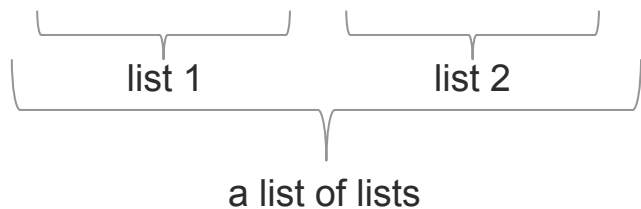
2D LIST

2D List

A list can be 2 or more dimensional.
2D lists are useful for representing a table of data.
Below shows an example of a 2D list.

example of a 2D list.

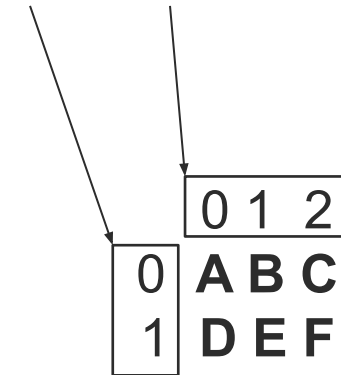
```
text = [ ['A', 'B', 'C'], ['D', 'E', 'F'] ]
```



accessing individual values

```
print( text[0][0], text[0][1], text[0][2] )  
print( text[1][0], text[1][1], text[1][2] )
```

`text[row][column]`



Screen output

```
A B C  
D E F
```

2D List

Creating a 2D list

How to create a list [["A", "A", "A"], ["A", "A", "A"]]?

Method 1: declare and assign

```
mylist = [ ["A", "A", "A"], ["A", "A", "A"] ]
```

Method 2: using for-loops

```
mylist = [ ["A" for i in range(3)] for j in range(2) ]
```

Method 3: using append

```
mylist = [ ]  
for row in range(2):  
    innerlist = [ ]  
    for col in range(3):  
        innerlist.append("A")  
    mylist.append(innerlist)
```

2D List

Assigning / editing values in a list

Similar to normal list, treat each item as a variable.

2D list

```
num = [ [10, 20, 30] , [ 30, 4 , 50] ]
```

editing a individual values

```
print(num[1][2])
```

```
num[1][2] = num[1][2] + 10
```

```
print(num[1][2])
```

Screen output

50

60

2D List

Iterating through the list using for-loop

Note that 2D list can have inner lists of *different lengths*.
Below shows how you display each item at a time.

```
# 2D list with inner lists of different lengths  
text = [ ["A", "B", "C"], ["D", "E"], ["F", "G", "H", "I"] ]
```

```
# iterate through each row  
for r in range ( len(text) ):  
    # for each row, iterate through the column  
    for c in range( len(text[r]) ):  
        print(text[r][c])
```

Screen output

```
A  
B  
C  
D  
E  
F  
G  
H  
I
```

2D List

Iterating through the list using for-loop

Below shows how you display a list in a table format.

2D list with inner lists of different lengths

```
text = [ ["A", "B", "C"] , ["D", "E", "F"] , ["G", "H", "I"] ]
```

iterate through each row

```
for r in range ( len(text) ):
```

for each row, iterate through the column

```
for c in range( len(text[r]) ):
```

```
    print(text[r][c] , end="")
```

```
print()
```

end="" prevents cursor going to next line
after this print command

go to new line after printing every row

Screen output

```
ABC
DEF
GHI
```

2D List

Iterating through the list using for-loop

This is an alternative shortcut to display a list in table format.

```
# 2D list with inner lists of different lengths
```

```
text = [ ["A", "B", "C"] , ["D", "E", "F"] , ["G", "H", "I"] ]
```

```
# iterate through each row
```

```
for r in range ( len(text) ):
```

```
    print(*text[r] , sep="")
```

← since each row is a list, display the row list
using * with **sep=""** (i.e. without space)

Screen output

```
ABC
DEF
GHI
```

DICTIONARY

DICTIONARY

Dictionary is a list of **key-value** pairs that enables quick access to values.

The keys must be unique as any repetition in key will be overwritten by the last occurrence.

example of a dictionary

```
dictionary = { "John": 15 , "Mary": 13 , "Bobby": 14 }
```

left side is the key
right side is the value

← note the *curly brackets* used

access individual value with key

```
print( dictionary[ "John" ] )  
print( dictionary[ "Mary" ] )  
print( dictionary[ "Bobby" ] )
```

note the *square brackets* used when accessing.
you cannot use *value* to search for *key*.
e.g. dictionary[15] is an error and will **not** give you "John"

Screen output

```
15  
13  
14
```



DICTIONARY

The keys and values of a dictionary can be of mixed data type.

Key must be a single data whereas value can be any data type including list.

example of a dictionary with mixed data type

```
dictionary = { "John": 15 , 13: "Mary" , 1.2: True , 15: ["age", 7] }
```

access individual value with key

```
print( dictionary['John'] )
```

```
print( dictionary[13] )
```

```
print( dictionary[1.2] )
```

```
print( dictionary[15] )
```

```
print( dictionary[15][0] )
```

*dictionary[15] is a list,
hence, [0] is the value at
index 0.*

Screen output

```
15
Mary
True
['age', 7]
age
```

DICTIONARY

If you need to declare many values in your dictionary, you can format it as shown below for better view.

example of proper formatting of dictionary with many values for clarity

```
dictionary = {  
    "John": 15,  
    "Mary": 13,  
    "Bobby": 14,  
    "Danny": 11,  
    "Smith Bob": 13,  
    "Marcus": 11,  
    "Marcus Lim": 12,  
}
```

DICTIONARY – Creating/Adding/Updating a dictionary

How to create a dictionary { "John": 15 , "Mary": 13 , "Bob": 14 } ?

Method 1: declare and assign

```
dictionary = { "John": 15 , "Mary": 13 , "Bob": 14 }
```

Method 2: using assignment

```
dictionary = { }  
dictionary[ "John" ] = 15  
dictionary[ "Mary" ] = 13  
dictionary[ "Bob" ] = 14
```

E.g. If "John" exists, it will be replaced, otherwise it will be added.
Hence, this method is also for adding and updating.

Method 3: using update()

```
dictionary = { }  
dictionary.update( { "John" : 15 } )  
dictionary.update( { "Mary" : 13, "Bob" : 14 } )
```

← You can add or update a set of values using this method

DICTIONARY - Editing values in a dictionary

You can perform various operations on each item the same way as a variable depending on its data type.

```
# dictionary variable
```

```
dictionary = { "John": 15 , "Mary": 13 , "Bob": 14 }
```

```
# example of editing a value in a dictionary
```

```
print( dictionary[ "John" ] )
```

```
dictionary[ "John" ] = dictionary[ "John" ] + 1
```

```
print( dictionary[ "John" ] )
```

Screen output

15

16

DICTIONARY - Deleting an item from a dictionary

Use ***pop()*** or ***del*** command to remove an item. ***pop()*** also returns the value of the popped item. You can use a variable to save the popped value (optional).

```
dictionary = { "John": 15 , "Mary": 13 , "Bob": 14 }
```

Method 1: Using ***pop()***

If you don't need the popped value then you do not need to use variable to save it

```
removed = dictionary.pop( "Mary" )  
print(dictionary)  
print(removed)
```

Method 2: Using *del*
del dictionary["Mary"]
print(dictionary)

Screen output

```
{'John': 15 , 'Bob': 14}  
13
```

Screen output

```
{'John': 15 , 'Bob': 14}
```

DICTIONARY – Looping through a dictionary

Dictionary does not have any index. You can only get a list of its keys and iterate through it.

Example

```
dictionary = { "John": 15 , "Mary": 13 , "Bob": 14 }
```

Method 1:

```
for key in dictionary:  
    print(key, dictionary[key])
```

Method 2:

```
for key, value in dictionary.items():  
    print(key, value)
```

This will take each key-value pair from the dictionary and place them in the variables key and value



Screen output

```
John 15  
Mary 13  
Bob 14
```



DICTIONARY – Looping through a dictionary

You can get a list of its keys or values using ***keys()*** and ***values()*** commands. Note that in Python's implementation, items in a dictionary *may not* always be stored in the order of its creation.

Example

```
dictionary = { "John": 15 , "Mary": 13 , "Bob": 14 }
```

```
keylist = list( dictionary.keys() )  
for i in range( len(keylist) )  
    print( keylist[i] )
```

cast the return value to a list so that you can manipulate it like a list.

```
valueslist = list( dictionary.values() )  
for i in range( len(valueslist) )  
    print( valueslist[i] )
```

Screen output

```
John  
Mary  
Bob
```

```
15  
13  
14
```



DICTIONARY

Checking key existence

You can check if a key exists in a dictionary using the ***in*** or ***not in*** operator.

Example

```
dictionary = { "John": 15 , "Mary": 13 , "Bob": 14 }
```

```
if "Mary" in dictionary:  
    print("Mary found")  
else:  
    print("Mary not found")
```

```
if "Sam" in dictionary:  
    print("Sam found")  
else:  
    print("Sam not found")
```

```
if "Kate" not in dictionary:  
    print("Kate not found")  
else:  
    print("Kate found")
```

Screen output

```
Mary found  
Sam not found  
Kate not found
```

FILE IO

read and write

WARNING

File IO can be a dangerous operation. If you are careless, you can delete or corrupt your files on the hard disk or you can create a file so huge that it exceeded your disk space and cause your computer to crash.

Hence, if your program doesn't seem to run as expected (eg. looping too long), terminate it immediately to check the codes.

Students caught performing malicious or mischievous acts will be suspended immediately and be subjected to disciplinary actions.

FILE INPUT OUTPUT

In order to save data permanently, you need to write them into files for subsequent retrieval. A file can be written in *text format* or *binary format*. A text format can be opened by any editor like note pad or word but a binary file requires interpretation to be readable. We will only be handling text format in this course.

To open a file for reading and writing the following command is used:

with open(*filename* , *mode*) as *myfile* :

eg. "myfile.txt"

file variable

Mode	Description
"r"	Read only (default). File must already exist.
"r+"	Read+Write. File must already exist.
"w"	Write only. Overwrites if file exists, otherwise creates a new file for writing.
"w+"	Read+Write. Overwrites if file exists, otherwise creates a new file for writing.
"a"	Appending. If file does not exist, new file will be created.
"a+"	Append+Read. If file does not exist, new file will be created.

FILE INPUT OUTPUT

Some commonly used commands for file operations are shown below.
We will focus on ***readline()*** and ***write()*** for subsequent examples.

Command	Description
read(n)	Read at most n characters from the file. Reads till end of file if n is negative or None.
readline(n)	Read and return one line from the file. Reads in at most n bytes if specified.
readlines(n)	Read and return a list of lines from the file. Reads in at most n bytes/characters if specified.
write(string)	Write a string to the file.
writelines(lines)	Write a list of lines to the file.
tell()	Returns the current file position (in bytes).
seek(position)	Go to file position (in bytes).

FILE INPUT OUTPUT

Writing

Example of creating and writing a file

with open("myfile.txt", "w") as f :

f.write("ABC")

f.write("DEF\n")

f.write("GHI\n")

↑ "w" sets file to write mode

← "\n" represents a newline character

A file named "myfile.txt" will be created and the contents are:

myfile.txt



ABCDEF

GHI

← the newline character is invisible in the file
but the next character will appear on the next line

FILE INPUT OUTPUT

Writing

For efficiency reason, a file write operation sometimes stays in the memory after the code is run and may only be executed after some amount of write operations is performed. To force the write operation to be executed immediately after the code, use ***flush()*** after the write command.

```
# Example of creating and writing a file
with open( "myfile.txt", "w" ) as f :
    f.write("ABC")
    f.flush()
```

FILE INPUT OUTPUT

Reading

Example of reading the file created in the previous slide

```
with open( "myfile.txt" , "r" ) as f :
```

```
    while True:
```

```
        text = f.readline()
```

```
        if text == "" :
```

```
            break
```

```
        else:
```

```
            print(text)
```

"r" sets file to read-only mode

the file will start from the top and read the next line when
readline() is used (it will remember its last location)

when text is "" (null), it means that the end of the file is reached

myfile.txt

```
ABCDEF
GHI
```

Each line read contains a
newline character so the cursor
will go to the next line. As print()
command itself adds another
newline, a blank row appears

Screen output

```
ABCDEF
GHI
```



FILE INPUT OUTPUT

Reading

rstrip() command can remove any newline or whitespace characters appearing at the end of a string.

Example of reading the file created in the previous slide

with open("myfile.txt" , "r") as f :

while True:

text = f.readline().**rstrip()** ← rstrip() will remove the newline character

if text == "" :

break

else:

print(text)

myfile.txt

```
ABCDEF
GHI
```

Screen output

```
ABCDEF
GHI
```

FILE INPUT OUTPUT

Appending

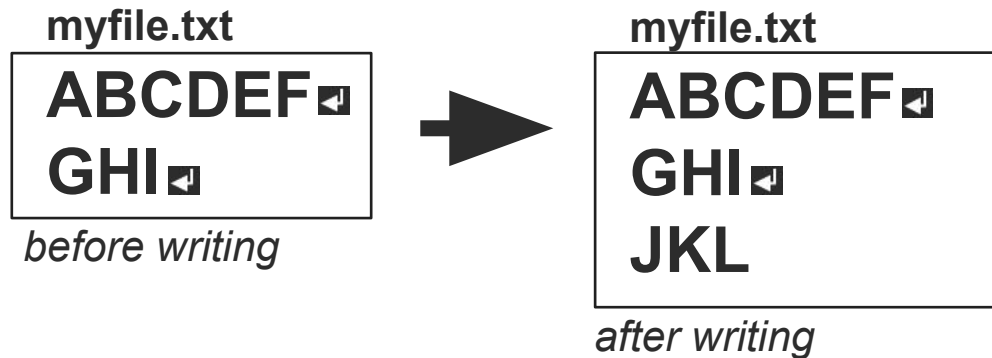
Use "a" mode to open a file for appending.

Example of appending the file created in the previous slide

with open("myfile.txt" , "a") as f :

f.write("JKL")

↑ "a" sets file to append mode
write() command will thus write to the end of file



FILE INPUT OUTPUT

Opening multiple files

You can open multiple files by nesting the open command. Below shows an example:

Example of opening multiple files together to copy the contents

```
with open( "file1.txt" , "r" ) as f1 :  
    with open( "file2.txt" , "w" ) as f2: } nest the open commands to keep both files opened  
        while True:  
            text = f1.readline()  
            if text == "":  
                break  
        else:  
            f2.write( text )
```

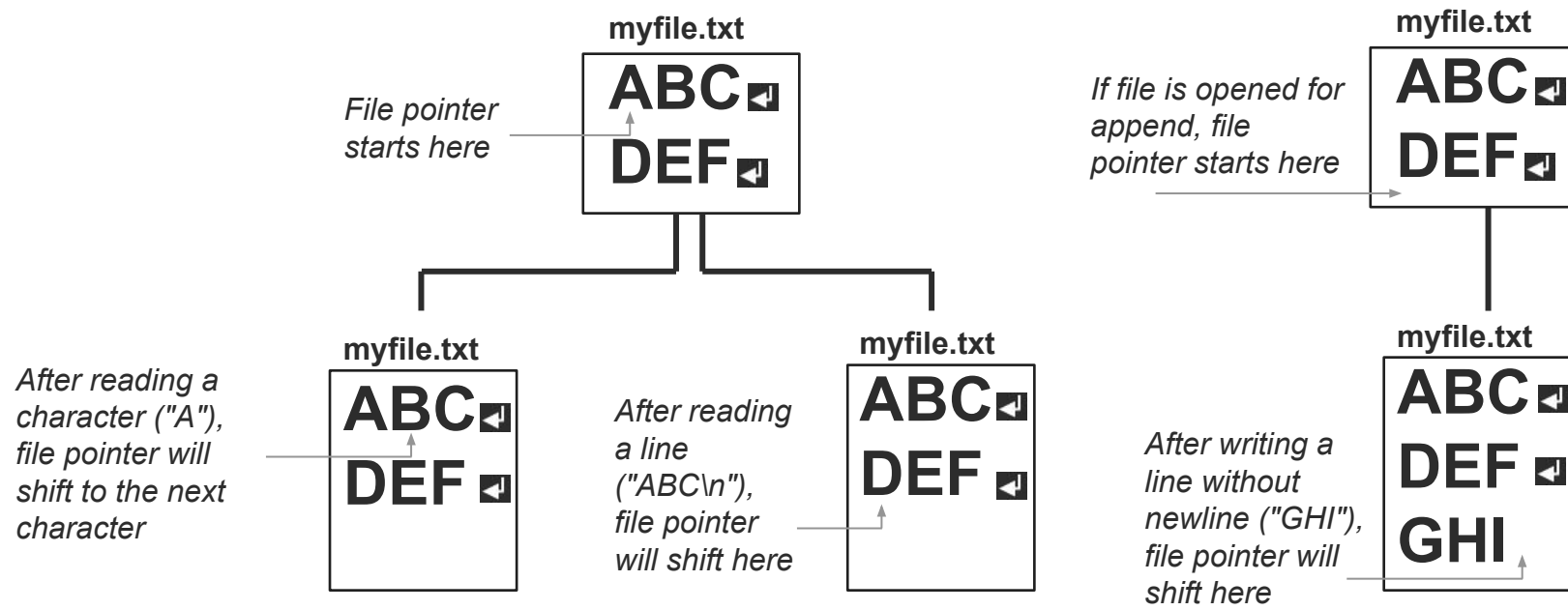
FILE IO

file pointer

FILE INPUT OUTPUT

File Pointer

When you open the file, there is a file pointer pointing to the beginning of the file. As you read or write, the file pointer changes to where your last character was read or written.



FILE INPUT OUTPUT

File Pointer

You can shift the position of a file pointer manually.

Use ***tell()*** to save the current pointer position into a variable. The value indicates the number of bytes (one character is one byte in ASCII code format)

```
position = f.tell()
```

Use ***seek()*** to shift the file pointer to a position

```
f.seek(position)
```

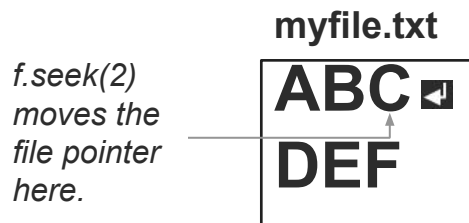
Note that any invisible characters such as the newline character will also be considered as a character when reading, writing and seeking.

FILE INPUT OUTPUT

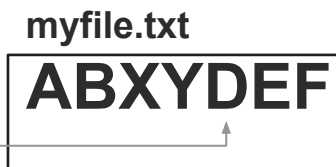
Replacing

A file is a series of bytes stored continuously on a hard disk. You are not able to insert a data between existing content or replace one string with a longer string without rewriting the file. A simple (but inefficient way) is to read the whole file, amend the contents and re-write the whole file with the updated content.

You can, however, replace a character with another character (or a string with another string with the same length). Below shows an example.



*f.write("XY")
Overwrites "C\n" and the
file pointer goes to the
next character ("D").
Note that the "DEF"
appears in the first row
as the newline character
is being replaced.*

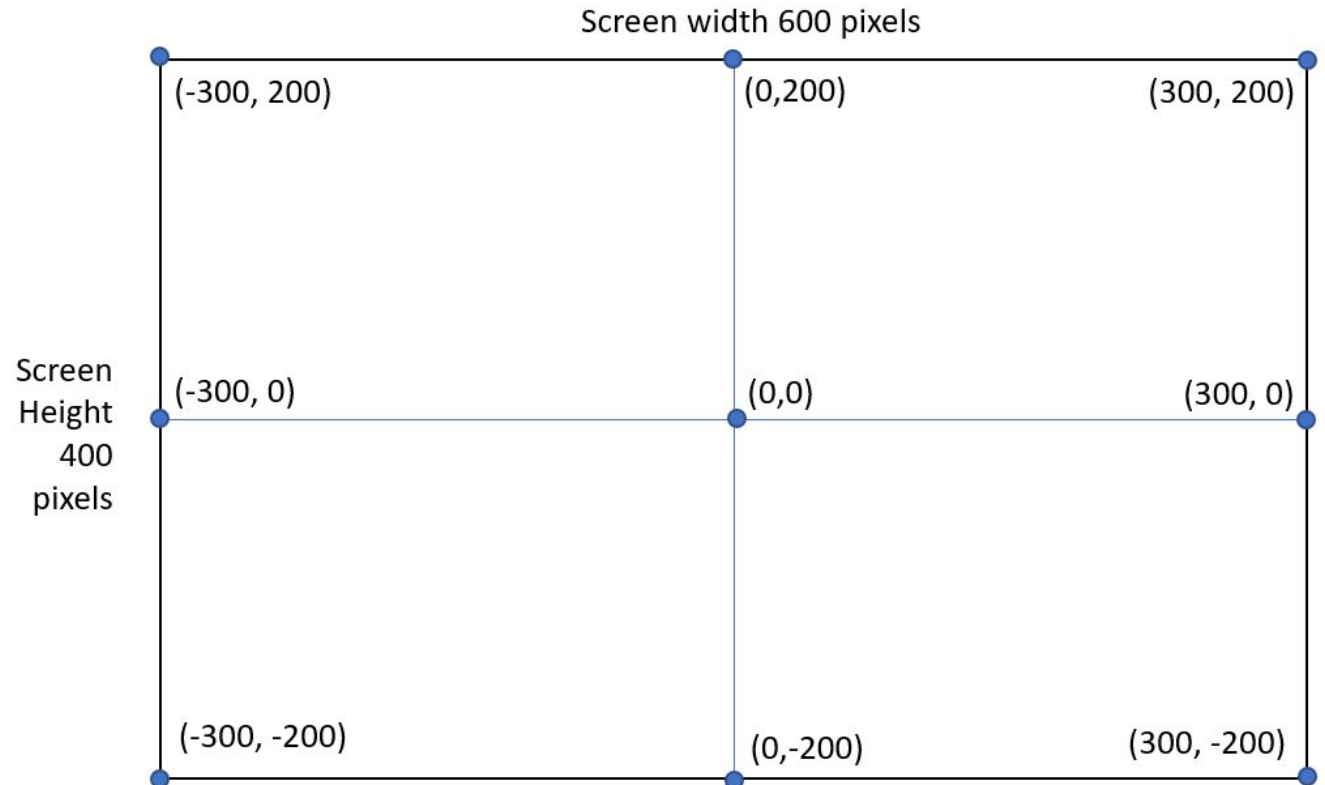


TURTLE

TURTLE

Turtle is a Python library that for creating graphics and text on a virtual canvas. The onscreen pen used for drawing is called the **turtle**.

The diagram on the right shows the coordinate system of turtle with a canvas width of 600 pixels by height 400 pixels.



TURTLE

To start a turtle program, there are 4 main steps:

1. Set up a canvas (the screen for drawing). Online IDE may have a default canvas without creating. But you will need to do this step in order to control the screen.
2. Set up a turtle pen. It is the cursor on the screen that draws as it moves. You can have more than one pen with different attributes (colour, line width, etc).
3. Start drawing with turtle commands.

The following slides will demonstrate with examples.

TURTLE

Step 1: Set up drawing canvas

Example

```
import turtle
```

```
# create a canvas
```

```
mymodule = turtle.Screen()
```

```
# set canvas size (width by height in pixels)
```

```
mymodule.setup( 400 , 300 )
```

```
# set the background colour
```

```
mymodule.bgcolor( "light blue" )
```

Screen output



TURTLE

Commands for Canvas Setting

Description	Command	Example (using variable 'myscreen')
Create a canvas (drawing screen)	<code>turtle.Screen()</code>	<code>myscreen = turtle.Screen()</code>
Set the screen size in pixel method 1: Using pixels method 2: Using ratio to desktop screen	<code>.setup(width , height)</code> <code>.setup(ratio , ratio)</code>	<code>myscreen.setup(400 , 400)</code> <code>myscreen.setup(0.5, 0.5)</code>
Set colour mode for RGB (1 or 255)	<code>.colormode(mode)</code>	<code>myscreen.colormode(1)</code> <code>myscreen.colormode(255)</code>
Set background colour of screen using name	<code>.bgcolor(colour name)</code> <code>.bgcolor(r , g , b)</code>	<code>myscreen.bgcolor("white")</code> <code>myscreen.bgcolor(25, 255, 79)</code>
Clear the screen	<code>.clear()</code>	<code>myscreen.clear()</code>
Clear the screen and reset the all settings to default	<code>.reset()</code>	<code>myscreen.reset()</code>
Force the screen to update after drawing if it doesn't	<code>.update()</code>	<code>myscreen.update()</code>
Returns the window width in pixel	<code>.window_width()</code>	<code>w = myscreen.window_width()</code> - <i>variable w is an integer</i>
Returns the window height in pixel	<code>.window_height()</code>	<code>h = myscreen.window_height()</code> - <i>variable h is an integer</i>



TURTLE

Step 2: Set up turtle pen

Example

continue from previous example

```
# create a turtle pen  
mypen = turtle.Turtle()
```

```
# set drawing colour  
mypen.color( "black" )
```

TURTLE

Commands for Pen Setting

Description	Command	Example (using variable <i>mypen</i>)
Create a pen. You can have multiple pens	<code>turtle.Turtle()</code>	<code>mypen = turtle.Turtle()</code>
Set the pen shape: square / arrow / circle / turtle / triangle / classic	<code>.shape(<i>shape name</i>)</code>	<code>mypen.shape("turtle")</code>
Set the pen stroke size in pixel	<code>.pensize(<i>size</i>)</code>	<code>mypen.pensize(5)</code>
Set the animation speed when the pen moves: 1 (slow) to 10 (fast) / 0 - instant	<code>.speed(<i>speed</i>)</code>	<code>mypen.speed(10)</code>
Set drawing colour method 1: using colour name method 2: using RGB (mode 255 - value 0 to 255)	<code>.color(<i>colour name</i>)</code> <code>.color(<i>r</i> , <i>g</i> , <i>b</i>)</code>	<code>mypen.color("green")</code> <code>mypen.color(150 , 12 , 122)</code>
Show pen on screen	<code>.showturtle()</code>	<code>mypen.showturtle()</code>
Hide pen on screen	<code>.hideturtle()</code>	<code>mypen.hideturtle()</code>
Set the pen down so that it will draw as it moves	<code>.pendown()</code>	<code>mypen.pendown()</code>
Set the pen up so that it will not draw as it moves	<code>.penup()</code>	<code>mypen.penup()</code>
Returns the pen position in screen coordinate	<code>.position()</code>	<code>x, y = mypen.position()</code> - <i>variables x and y are floats</i>
Returns the direction of the pen in degree	<code>.heading()</code>	<code>degree = mypen.heading()</code> - <i>variable degree is a float</i>



TURTLE

Step 3: Draw with commands

Example

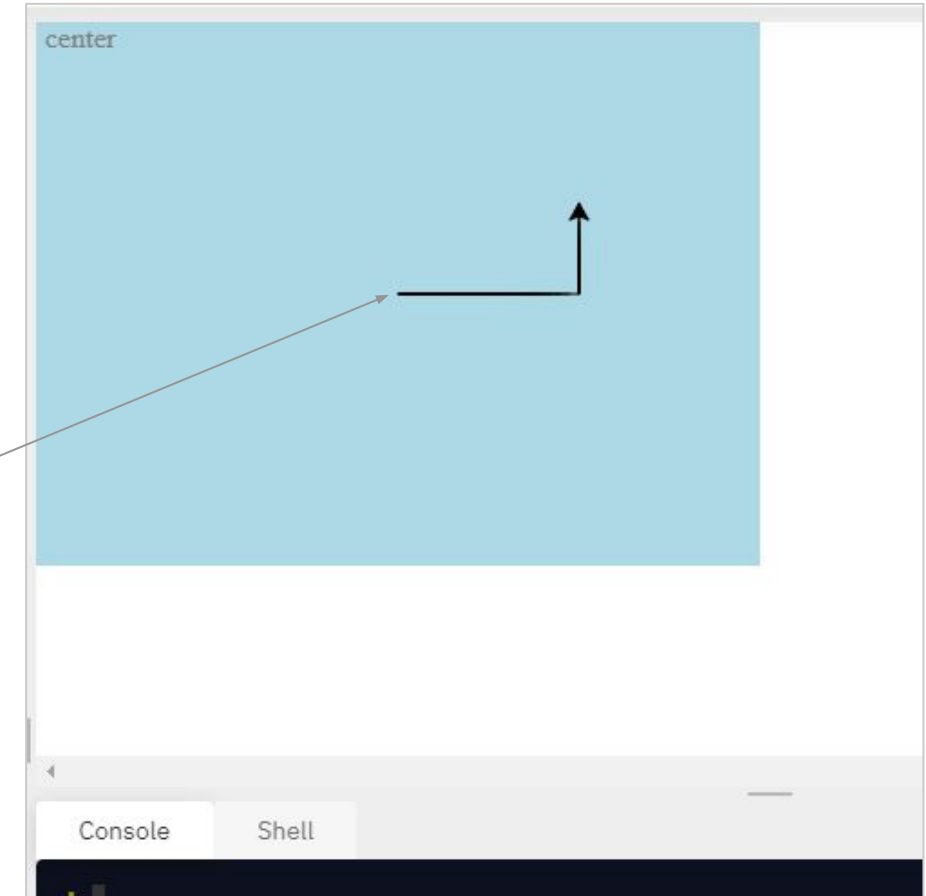
continue from previous example

```
# create a turtle pen  
mypen.forward( 100 )  
mypen.left( 90 )  
mypen.forward( 50 )
```

```
# (OPTIONAL) force the screen to update if it doesn't  
# this may happen on certain IDE when using input() command  
myscreen.update()
```

the pen starts at the centre of
the canvas, which is (0, 0)

Screen output



TURTLE

Commands for Drawing

Description	Command	Example (using variable <i>mypen</i>)
Move pen forward	<code>.forward(<i>pixel</i>)</code>	<code>mypen.forward(150)</code>
Move pen backward	<code>.backward(<i>pixel</i>)</code>	<code>mypen.backward(100)</code>
Move pen to a specific point stated	<code>.goto(<i>x</i> , <i>y</i>)</code>	<code>mypen.goto(200, -300)</code>
Move pen to a specific x coordinate	<code>.setx(<i>pixel</i>)</code>	<code>mypen.setx(-400)</code>
Move pen to a specific y coordinate	<code>.sety(<i>pixel</i>)</code>	<code>mypen.sety(400)</code>
Turn pen to its left by certain degree (anti-clockwise)	<code>.left(<i>degree</i>)</code>	<code>mypen.left(17)</code>
Turn pen to its right by certain degree (clockwise)	<code>.right(<i>degree</i>)</code>	<code>mypen.right(400)</code>
Turn pen to a specific degree	<code>.setheading(<i>degree</i>)</code>	<code>mypen.setheading(45)</code>
Move pen in a circle with stated radius in pixel	<code>.circle(<i>radius</i>)</code>	<code>mypen.circle(50)</code>
Move pen in an arc with stated radius and degree	<code>.circle(<i>radius</i> , <i>degree</i>)</code>	<code>mypen.circle(100 , 180)</code>
Move pen in a polygon with specified number of sides. The polygon is within an arc with stated radius and degree.	<code>.circle(<i>radius</i> , <i>degree</i> , <i>sides</i>)</code>	<code>mypen.circle(100 , 180 , 3)</code>



TURTLE

In Turtle, you cannot fill a shape that has already been drawn. You must indicate that you are going to perform filling before drawing the shapes.

There are 4 steps for filling shapes with colours:

1. Set outline and fill colours (this overwrites existing pen colour) using **.color()** and **.fillcolor()**
2. Enable fill using **begin_fill()**
3. Draw all the shapes that you want to be filled with these colours. Whenever a closed shape is formed, it will be filled.
4. Disable fill when you are done using **end_fill()** and all closed shapes will be filled.

Commands for Filling

Description	Command	Example (using variable <i>mypen</i>)
Set outline colour	.color(outline colour)	mypen.color("black")
Set fill colour	.fillcolor(fill colour)	mypen.fillcolor("yellow")
Start the filling process	.begin_fill()	mypen.begin_fill()
End the filling process	.end_fill()	mypen.end_fill()



PYTHON CODING

TURTLE

TURTLE

Example

```
import turtle
```

```
# create a canvas
```

```
mymScreen = turtle.Screen()  
mymScreen.setup( 400 , 300 )  
mymScreen.bgcolor( "light grey" )
```

```
# create pen
```

```
mypen = turtle.Turtle()  
mypen.pensize(6)
```

```
# set draw a circle with blue stroke and fill it with yellow
```

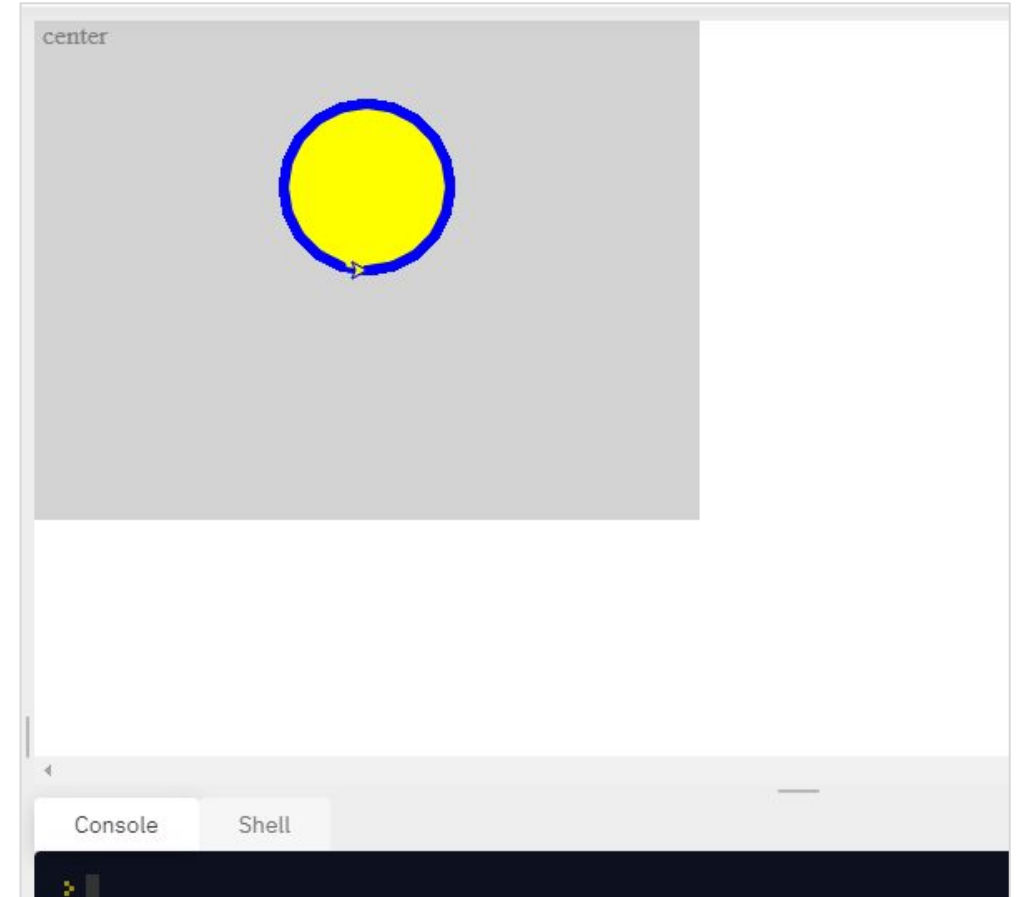
```
mypen.color( "blue" )  
mypen.fillcolor( "yellow" )
```

```
mypen.begin_fill()
```

```
mypen.circle(50)
```

```
mypen.end_fill()
```

Screen output



TURTLE

You can also place text on turtle canvas. You need to set the text style when calling the write() function to place the text.

Commands for Text

Description	Command	Example (using variable <i>mypen</i>)
Set the font style	.write(text, font, align)	<pre>style = ("Arial" , 30 , "bold") - set font, size and attribute and save to a variable pen.write("Hello!" , font=style , align="center") - call the write function to place the text</pre> <pre>Font (examples) : Courier / Arial / Calibri / Times / Comic Sans MS / Monoton / Calligraphr Attribute : normal / bold / italic / bold italic Align : left / center / right</pre>

TURTLE

Colour reference

Name	Red	Green	Blue
AliceBlue	240	248	255
AntiqueWhite	250	235	215
aquamarine	127	255	212
azure	240	255	255
beige	245	245	220
bisque	255	228	196
black	0	0	0
BlanchedAlmond	255	235	205
blue	0	0	255
BlueViolet	138	43	226
brown	165	42	42
burlywood	222	184	135
CadetBlue	95	158	160
chartreuse	127	255	0
chocolate	210	105	30
coral	255	127	80
CornflowerBlue	100	149	237
cornsilk	255	248	220
cyan	0	255	255
DarkBlue	0	0	139
DarkCyan	0	139	139
DarkGoldenrod	184	134	11
DarkGray	169	169	169
DarkGreen	0	100	0
DarkGrey	169	169	169
DarkKhaki	189	183	107
DarkMagenta	139	0	139
DarkOliveGreen	85	107	47

Name	Red	Green	Blue
DarkOrange	255	140	0
DarkOrchid	153	50	204
DarkRed	139	0	0
DarkSalmon	233	150	122
DarkSeaGreen	143	188	143
DarkSlateBlue	72	61	139
DarkSlateGray	47	79	79
DarkSlateGrey	47	79	79
DarkTurquoise	0	206	209
DarkViolet	148	0	211
DeepPink	255	20	147
DeepSkyBlue	0	191	255
DimGray	105	105	105
DimGrey	105	105	105
DodgerBlue	30	144	255
firebrick	178	34	34
FloralWhite	255	250	240
ForestGreen	34	139	34
gainsboro	220	220	220
GhostWhite	248	248	255
gold	255	215	0
goldenrod	218	165	32
gray	190	190	190
green	0	255	0
GreenYellow	173	255	47
grey	190	190	190
honeydew	240	255	240
HotPink	255	105	180

Name	Red	Green	Blue
IndianRed	205	92	92
ivory	255	255	240
khaki	240	230	140
lavender	230	230	250
LavenderBlush	255	240	245
LawnGreen	124	252	0
LemonChiffon	255	250	205
LightBlue	173	216	230
LightCoral	240	128	128
LightCyan	224	255	255
LightGoldenrod	238	221	130
LightGoldenrodYellow	250	250	210
LightGray	211	211	211
LightGreen	144	238	144
LightGrey	211	211	211
LightPink	255	182	193
LightSalmon	255	160	122
LightSeaGreen	32	178	170
LightSkyBlue	135	206	250
LightSlateBlue	132	112	255
LightSlateGray	119	136	153
LightSlateGrey	119	136	153
LightSteelBlue	176	196	222
LightYellow	255	255	224
LimeGreen	50	205	50
linen	250	240	230
magenta	255	0	255
maroon	176	48	96



Colour reference

[illegible]